BuoyUp — Technical Report

Oct 18, 2026 · @dungtt

1. Summary

BuoyUp is a working lending platform in which five AI agents take an e-commerce seller from "paste my shop link" to money in the account, while a fixed credit algorithm and a FISCO BCOS ledger keep every agent accountable. It targets sellers on Chinese marketplaces (Taobao/Tmall, JD, Pinduoduo, Douyin, Kuaishou) who have steady platform revenue but little or no bank credit history.

The design rests on one rule: **agents choose, explain and check; they never create a money figure.** Every limit, rate, grade and repayment schedule comes from a deterministic algorithm (blocks B1–B12). Every pass/fail verdict an agent proposes is re-checked by code. Every step of a dossier is fingerprinted and anchored on a 4-node FISCO BCOS chain, run by the team's own Go service and Solidity contracts.

What the system does end to end:

- **Verify** (Agent 1): proves shop ownership, reads tax-declared revenue, credit report, interbank (FATE) score and ID card (OCR), and raises risk flags.
- **Advise** (Agent 2): turns the algorithm's limit into options, recommends borrowing "just enough", and chats with the seller.
- **Control** (Agent 3): an independent two-phase challenge with its own challenger model and hybrid legal retrieval over 794 Chinese statute articles; seals the dossier on-chain.
- **Syndicate**: WeBank leads; partner banks decide in parallel within 30 seconds; capital is split 40/30/30, 30/70 or stepped down to 70% / 50%.
- **Disburse and monitor** (Agent 4a/4b): one-time payout grant, encrypted per-bank withdrawal orders on-chain, automatic collection, three early-warning alerts.
- **Post-audit** (Agent 5): compares decisions with outcomes, writes seller reports, and proposes parameter changes that a second human must approve.

Current evidence: 467 passing backend tests, 31 attack cases on authentication, credit tampering, LLM and legal citations and multi-bank authorization, and a 50-dossier end-to-end run that went from 47/50 to 50/50 after three weaknesses it found were fixed (Section 9). The whole service runs as one FastAPI process with a SQLite file, and keeps working (template wording) when the LLM is down.

2. Problem and goals

Small online sellers need working capital weeks before a sales peak (11.11, 6.18), but banks cannot see the platform data that proves their business. Since the 2021 "断直连" rule, marketplaces may not pass user data straight to lenders, so a bank must assemble evidence from lawful channels: what the seller submits, tax data (银税互动), the credit bureau, its own account flows, and privacy-preserving queries to partners (FATE).

The competition theme, *AI as a Financial Participant*, asks for agents that act, not chatbots. Lending is a domain where an agent's action moves real money, so the hard part is control: an LLM that invents a limit, follows an instruction hidden in an uploaded document, or pays out twice causes direct loss.

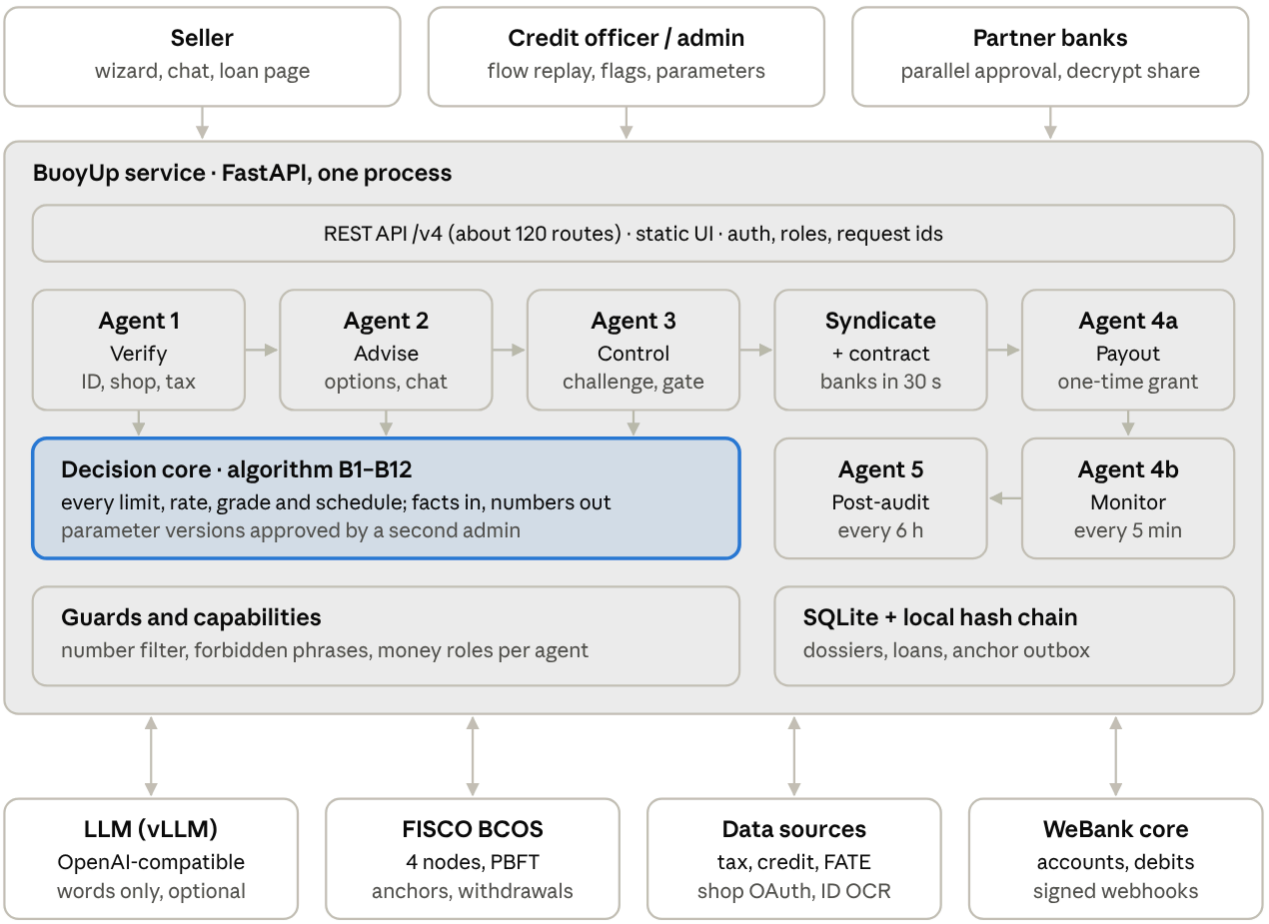
BuoyUp sets five goals:

Goal	How it is met
Serve thin-file sellers	Revenue from tax invoices and verified statements; a capped "seed loan" for first-time borrowers; thin-FATE fallback
Decide in minutes, not days	Agents run the full pipeline; partner banks approve in parallel within 30 s
Agents never touch the numbers	Fixed algorithm B1–B12 computes every money figure; code filters every LLM output
Every step provable	Hash-chained audit anchors on FISCO BCOS; dossier can be re-verified by any bank
Fail safe	Errors route to a human; LLM outage falls back to templates; chain outage blocks approval rather than skipping the seal

Running example used across the code and tests: **Ms. Lan** (女装 seller on Douyin, household business, tax revenue ≈166,700 CNY/month, grade B). The fully fictional Tmall shop "Nhã Cầm Gia Dụng" (周雅琴) is a second, clean-profile sample.

3. System architecture

BuoyUp runs as a single FastAPI/uvicorn process that serves both the `/v4` REST API and the static web UI, with one SQLite file for state. No Node, Redis or Postgres is needed to run it, and Docker Compose deploys it as one container plus a data volume.



BuoyUp v4 architecture · one service, four external systems

The top row of agents is the loan pipeline from left to right; Agents 4b and 5 run in a background autopilot loop for the life of each loan. Agents 1–3 pass verified facts down to the decision core and read numbers back; they never write figures themselves.

Package (`BackEnd/src/app`)	Responsibility
`core/`	Algorithm B1–B12, parameter set, legal rules and corpus, RAG, challenger model, flow replay
`agents/`	Agents 1–5, contract, complaints, LLM tool loop, notices
`banks/`	Bank eligibility, B12 split, parallel syndicate round
`blockchain/`	Per-case hash chain, remote FISCO client, catalogue sync, encrypted withdrawals
`data_sources/`	Six data sources, sample profiles, consent, shop connection

Package (`BackEnd/src/app`)	Responsibility
`autopilot/`	Background loop for Agents 4b and 5 and seller reports
`platform/`	Config, database tables, security, capabilities, guards, audit log, LLM clients
`Reid/` , `CCCD/`	Face authentication (InsightFace) and ID card OCR (RapidOCR)

Code size: about 21,600 lines of Python in the application package and 10,800 lines of HTML, CSS and JavaScript (excluding vendored libraries).

4. The five agents

Each agent has a narrow tool set, a fixed role in the pipeline, and a deterministic code path that runs when the LLM is absent or its output is rejected. The table separates what the model does from what code does.

Agent	Stage	Model's part (LLM)	Code's part (authoritative)	Money privilege
1 · Verify (`agent1_verify.py`)	Shop link → verified facts	Order of tool calls, wording of follow-up questions (numbers frozen), notes; flags text aimed at the reviewer	Shop ownership proof, ID card OCR + GB 11643 checksum + age + expiry + name match, tax revenue, credit, FATE, statement check (±15% tolerance, >30% → reject); opens and closes B10 flags	None
2 · Advise (`agent2_advise.py`)	Limit sheet → options	Chooses among sheet options, explains factors, chats, what-if	`propose` accepts only an `option_id` already on the sheet; slider goes straight to code; number filter and threshold filter on every message	None

Agent	Stage	Model's part (LLM)	Code's part (authoritative)	Money privilege
3 · Control (`agent3_control.py`)	Independent challenge + gate	Phase-1 judgment on three judgment rules, temperature 0, own session, optionally a different model	28 rules (law, policy, risk, AML, goods), floor verdict, challenger model, on-chain seal, five-condition gate	None
4a · Payout (`agent4a_payout.py`)	Signed contract → disbursement	Bank-split choice and seller notice (checked by code)	One-time grant bound to application, amount and payee, valid 15 min, revoked after use	Pay out once
4b · Monitor (`agent4b_monitor.py`)	Life of the loan	Notice wording	Daily tick: due-date debit (penalty → interest → principal), reminders 7 and 1 days ahead, 3-day grace, 30-day bureau report, B11 alerts	Read-only; debits run as the borrower's standing order
5 · Post-audit (`agent5_*.py`)	After loans mature	Summaries and proposals	Prudence backtest, outcome statistics, monthly/quarterly/annual seller reports	Read-only; can only propose parameters

How Agent 3 stays independent

Agent 3 is the safety core, so it is built to disagree with Agent 2 rather than echo it.

1. **Phase 0 (code):** every machine-checkable rule runs and sets a *floor*, the strictest verdict any violated rule demands.
2. **Phase 1 (blind):** Agent 3 judges the case without seeing Agent 2's rationale. A logistic challenger model scores default risk on features that share nothing with the scorecard (declared vs invoice price, supplier links, inventory vs tax returns, sales spikes, B2B share). PD ⩾0.30 pushes toward a smaller amount, ⩾0.60 escalates. The phase-1 verdict is sealed on-chain before phase 2 may start.
3. **Phase 2 (debate):** it reads Agent 2's rationale and may question it for at most 2 rounds. It may only go lighter than phase 1 when each resolved concern points to evidence that exists, and never below the floor.

4. **Gate (code, trusts no agent):** approval needs all five conditions: dossier verified and every closed flag backed by a successful tool call; no hard-reject rule and floor = approve; Agent 2 and 3 agree; the seal is confirmed on-chain; amount under the 500,000 auto-limit, no human request, and the auto-approval kill switch on (it switches off when a segment's bad-loan ratio passes 8%).

Legal grounding uses hybrid retrieval over 794 articles from 14 Chinese laws and measures (AML Law 2024, E-Commerce Law, PIPL, internet loan measures, prohibited goods list…): BM25 on Chinese character bigrams plus multilingual-e5-large embeddings, fused by reciprocal rank fusion. On a 22-question hold-out set, recall@5 rises from 0.59 (BM25) to 0.82 (hybrid). Every rule's citation must match the article text verbatim, or the rule is disabled at load time.

Orchestration around the agents

- **Contract (code only):** the borrower picks an amount within the approved limit and a due day (1–28); the cost disclosure must stay on screen 20 seconds; signing by face or OTP; bank capacity is reserved at signing.
- **Complaints:** acknowledged in 1 working day, answered with reasons in 5, at most 15.
- **Autopilot:** Agent 4b runs every 5 minutes and on every inflow; Agent 5 every 6 hours; due seller reports are looked for every hour.

5. Core credit algorithm (B1–B12)

The credit core is a set of pure functions in `app/core/algorithm.py`: the same inputs always give the same numbers, money is integer yuan, and Python's half-even rounding is used on purpose so the browser can reproduce every figure. Agents feed it verified facts and read its output; none of them can pass it a free amount.

Block	Computes	Rule (parameter set v4.0)
B1 Customer group	1 good history in network · 2 credit elsewhere · 3 never borrowed · 4 late/default	≥30 days late, default or NPL in 24 months → group 4
B3 Capital need	Plan value minus inventory, own capital and other loans	Unit price = min(declared, invoice price); no invoice → 15% haircut; new supplier × 0.70
B4 Prudent revenue	Monthly revenue the loan may lean on	12-month tax average × (1 + growth, capped 50%) × industry prudence (0.65–0.80); verified statements +0.05

Block	Computes	Rule (parameter set v4.0)
B5 Repayment cap	Cost-of-goods share of each yuan of revenue	(1 − gross margin) ÷ (1 − platform fee)
B6 Scorecard → grade	Nine indicators, 100 points	A ⩾85 · B ⩾70 · C ⩾55 · D ⩾40 · else E; ⩾90 days late → hard E
B7 Rate	Rate by grade, above a cost floor, below the legal cap	A 8% · B 10% · C 12% · D 15% · E rejected (no risk pricing up); cap 24%
B8 Limit	Smallest of the caps below, then the seed ratio	Rounded down to 100; minimum loan 10,000
B9 Schedule	Equal-principal instalments, interest by actual days (365)	Minimum payment = interest + 30% of one instalment
B10 Risk flags	Inflow gap, pass-through, spikes, price deviation, ID mismatch	Raised by Agent 1, closed only by evidence or an officer
B11 Early warning	Weighted post-loan signals → green / yellow / red	Yellow ⩾0.2, red ⩾0.5 (Agent 4b)
B12 Bank split	Shares among syndicate banks	Each partner ⩾30%; per-bank caps read from the on-chain catalogue

The limit in B8 is the binding minimum of up to four caps:

$$\text{limit} = \left\lfloor \min\left(\frac{R_p \cdot T \cdot s}{1 + r\,T/12},\; m_g \cdot \bar{R},\; C_{type},\; C_{thin} \right) \right\rfloor_{100} \times \rho_{seed}$$

Here R_p is prudent monthly revenue (B4), T the longest tenor, s the usable repayment share (B5), r the annual rate (B7), m_g the group multiplier (3.0 / 1.5 / 0.5 / 0), $\bar{R}$ the 12-month average revenue, C_{type} the borrower-type cap (individual 50,000 · household business 200,000 · company 2,000,000) and $C_{thin} = \min(0.5 \times \bar{R}, 50{,}000)$ when the FATE signal is thin.

Two safeguards protect the bank on first loans:

- **Seed loan.** The first loan is 50% of the computed limit, then 75% and 100%. The step rises only after ⩾3 months in which inflows to the WeBank business account reach ⩾80% of tax revenue, so a good repayment record alone cannot inflate the limit.
- **Thin FATE.** When the interbank score rests on fewer than 2 banks, its 15 scorecard points are rebuilt from signals the scorecard does not already use (invoice regularity, supply chain, payroll, registry age) and the C_{thin} cap applies.

Every reduction is explained with a signed reason code (`PRICE_FROM_INVOICE`, `GROUP_CAP`, `THIN_FATE_CAP`, `SEED_LOAN`…) whose impacts add up from the requested amount to the final limit.

Worked example, Ms. Lan: prudent revenue 166,700 × 1.20 × 0.70 = 140,028/month. Caps: affordability 962,374 · group 250,050 · type 200,000 → computed limit 200,000; first loan at 50% → 100,000 at grade B, 10% a year.

Parameter governance. Each value in `core/params.py` is tagged with its origin: precedent (`TIEN_LE`), team decision on an open point (`DOI_CHOT`) or hand-set and awaiting data (`DAT_TAY`). Business code reads only the active version; Agent 5 may propose a change, a different admin must approve it, and each approved version is hashed onto the chain.

6. Bank syndicate and FISCO BCOS ledger

BuoyUp spreads each loan across several banks and records every step on a shared FISCO BCOS chain, so no single party, BuoyUp included, can quietly change a decision or a payment.

Parallel bank approval (B12)

Once Agent 3's gate approves, the orchestrator (`banks/syndicate.py`) sends the case to all eligible partner banks at once. Before asking, it enforces hard rules: partner on the whitelist, local banks only in their own province, monthly budget left, and the joint-lending concentration limits for internet loans (≤25% of tier-1 capital, ≤50% of total loans). Each bank then applies its own on-chain policy (minimum shop rating and age, rate band, maximum tenor, maximum amount per loan) and must answer within 30 seconds; a late answer counts as a decline.

Banks that agree	Split	Fallback
Two or more partners	WeBank 40 / first two partners 30 / 30	—
One partner	WeBank 30 / partner 70	—
None	Retry at 70% then 50% of the limit	WeBank alone up to 50,000, else reject

Shares are computed in basis points on integers, so "total exactly 100.00%" is checked without floating-point error. Each partner keeps at least 30%. The tenor offered to the borrower is capped by the shortest maximum tenor among participating banks. Example

from the live chain catalogue: Ms. Lan (rating 4.6, 26 months) is declined by CMB (needs 4.7) and ICBC (needs 36 months); CCB agrees, giving WeBank 30 / CCB 70 with a 3-month tenor because CCB lends at most 120 days. The approved limit stays valid for 7 days.

What goes on-chain

The chain runs as a FISCO BCOS service with 4 nodes (PBFT, group 1), a public explorer and a REST API. BuoyUp uses it in three ways:

Use	What is written	Endpoint
Audit anchors	Salted SHA-256 fingerprint of each dossier step, hash-chained per case: consents, verification record, limit sheet, Agent 3 seal, debate rounds, gate decision, contract, payout, each debit, seller reports, parameter versions, legal corpus hash	`POST /v1/audit/anchors`, `GET …/verify`
Encrypted withdrawal orders	Per-bank envelope: one AES-256-GCM piece per bank, keys wrapped with X25519 + HKDF-SHA256, Merkle root over the piece fingerprints; the public view never shows a bank name	`POST /v1/withdrawals`
Bank catalogue	Limits, policies, contracts and public keys of 10 banks, synced at start-up	`GET /v1/banks`, `/v1/overview`

Each bank decrypts only its own piece and can prove it to an arbiter without revealing its key, because the salt lives inside the ciphertext. A bank-side standalone script (`static/decrypt_withdrawal.py`) does the decryption outside BuoyUp.

The chain service (`sgchain`, Go)

The FISCO BCOS side is a separate service the team built (github.com/ffleader1/fisco-bcos-verifier, `fisco-bcos-verifier`: about 6,400 lines of Go and 3 Solidity contracts). BuoyUp reaches it only over REST at `https://fs.buoyup.app`, so the chain rules hold even if the lending service misbehaves.

Item	Choice
Chain	FISCO BCOS v2.11.1, 4 sealer nodes, group 1, PBFT consensus, ECDSA keys
SDK	FISCO BCOS go-sdk v1.1.1 (pure Go)
Contracts	Solidity 0.6.10; compiled ABI and bytecode embedded in the binary
Ledgers	`fisco` (live chain) or `mock` (in-memory, same rules) behind one `Ledger` interface
CLI	`keygen` · `deploy` · `bootstrap` · `serve` · `decrypt`
Pages	REST API with Swagger (`/docs/`), public explorer (`/explorer/`), team overview (`/overview/`)

Smart contracts. The rules that matter most are enforced on-chain, not only in Go:

Contract	What it enforces
`Roles`	Bitmask roles: ADMIN deploys and grants; OPERATOR (the BuoyUp service key) writes withdrawals; AUDIT_WRITER anchors hashes. An admin cannot revoke its own admin role. Stable revert codes (`E_UNAUTHORIZED` , `E_NOT_FOUND` , `E_INVALID` , `E_DUPLICATE`) map to HTTP errors
`WithdrawalLedger`	One record per withdrawal id (a duplicate is rejected); 2–21 pieces (summary + up to 20 banks); envelope ≤32 KB; no empty commitment; the contract recomputes the SHA-256 Merkle root of the commitments and rejects any record whose root does not match
`AuditTrail`	11 record kinds (application, algorithm output, agent I/O, credit memo, risk challenge, human decision, offer and contract, PIPL Art. 24 explanation, debit notice, decision package, other). Per subject, head = keccak256(previous head ‖ kind ‖ content hash ‖ timestamp), so deleting or reordering an off-chain record is detectable. The same content cannot be anchored twice for one subject; `locate` finds where a hash first appeared

Service-side checks before anything reaches the chain:

- **Bank split:** each ratio above 0 with at most 2 decimals, total exactly 100.00%, every bank known, in the request's region and under an active contract; otherwise `400` with `invalid_bank_ratio`, `unknown_bank`, `bank_region_mismatch` or `bank_unavailable`. Per-bank amounts are amount × ratio, with the largest share taking the rounding remainder so the legs always add up.
- **Actor:** as an agent, only `agent-4` with its version may withdraw; a human withdrawal needs staff id, role and a reason. The chain stores the actor type, a salted staff reference and the hash of the full actor object.
- **Privacy:** user id and account number become a salted `accountRef`; the plaintext never goes on-chain.
- **Idempotency:** a repeated `requestId` returns the original withdrawal instead of paying again. The payout itself is a mock treasury behind a `payout.Payer` interface that a real payment system replaces.
- **Verification:** `GET /v1/withdrawals/{id}/verify` recomputes the Merkle root, decrypts every piece, matches the fingerprints, re-checks the 100% split and asks every node whether it holds the transaction.
- **Bank catalogue:** 10 demo banks (VN: Vietcombank, Techcombank, BIDV, Agribank, MB Bank; CN: WeBank, ICBC, CCB, BOC, CMB) in YAML, each with numeric limits, a written credit policy for Agent 3 and a mock contract, both hashed with SHA-256. Unknown YAML fields fail start-up. All bank data is mock data for the demo.

Tests: Go unit tests (hashing, catalogue, sealing), end-to-end API tests, and a ledger conformance suite that runs on the mock with `go test ./...` and, with `make test-integration`, on a live 4-node chain with fresh contracts and keys.

Live deployment, checked on 18/10/2026: `https://fs.buoyup.app/v1/chain` reports block 334 with 4 of 4 sealer nodes online; `AuditTrail` is deployed at `0xc2986f90e429e46c3df094a5ff99cdd184a90ac1` and `WithdrawalLedger` at `0x14d6585bbfd8883e1916189c16a59a1bd5bb3e23`.

Failure behaviour

- **Non-critical anchors** that fail are queued in an outbox and retried every 5 minutes (up to 20 attempts).
- **Critical anchors** (Agent 3 phase-1 seal, Agent 3 conclusion, gate decision, contract signed) fail the request with `503 CHAIN_UNAVAILABLE` and roll back, so no case is approved without its seal.
- **Payouts** are idempotent: `requestId = disburse:<case id>` means a retry can never pay twice.

- **Local mode** (`BUOYUP_CHAIN_BACKEND=local`) keeps a real per-case hash chain with tamper detection in the database, simulating only the four nodes; it lets the system and its tests run offline.

7. Security and compliance design

Security in BuoyUp is enforced where money moves and where models speak, not only at the login screen. Four layers stack: who may act, what an agent may do, what a model may say, and what can be proven afterwards.

Permission tiers

Actor	Can read	Can change	Cannot
Seller	Own dossiers, loans, notices, reports	Apply, sign consent and contract, repay, prepay, complain	See other sellers (IDOR returns 403/404), pick tenor or rate
Credit officer	All dossiers, Agent 3 trace, full B1–B9 flow	Resolve flags, escalated decisions, re-authorize a payout	Edit parameters alone
Admin	Everything above, chain status	Approve a parameter version proposed by someone else	Approve own proposal
Partner bank	Its parallel decisions, its own withdrawal pieces, on-chain terms	Decide its share in the syndicate round	Decrypt another bank's piece
`agent4a`	Contract, grant	One payout, with a 15-minute grant bound to case, amount and payee	Debit, pay twice
`standing_order`	Loan, account	Debit the WeBank business account on due dates and on inflow	Pay out

Actor	Can read	Can change	Cannot
`agent4b`, `agent5`	Loans, accounts, reports	Score, alert, notify, write reports	Any payout or debit (raises `CapabilityDenied`)

The capability check sits inside the functions that move money (`platform/capabilities.py`), so a bug in routing cannot grant a monitoring process the power to pay.

Controls on the language model

- **No free amounts.** No tool in any agent accepts a loan amount; Agent 2 can only pick an `option_id` from the sheet.
- **Number filter.** An outgoing message may contain only numbers present in the sheet or a tool result; internal thresholds are blocked.
- **Forbidden phrases.** Promises of approval, disbursement or lower rates, threats and term changes are listed per agent role (`platform/guards.py`). A hit discards the whole output and uses the template; it is never repaired.
- **Prompt injection.** Uploaded documents feed only fixed fields; text aimed at the reviewer is recorded (RISK-06) and escalates the case. TestBench cases DS-38 (injection via chat) and DS-39 (via data) confirm limit and proposal do not move.
- **Independent reviewer.** In production, the server refuses to start if Agent 3′s model equals Agent 2′s (`LLM_AGENT3_MODEL_NAME` must differ).
- **Graceful failure.** Timeout, bad JSON or schema mismatch returns `None`; the agent continues on its deterministic path.

Identity, consent and ownership

- **Passwords and sessions:** PBKDF2-HMAC-SHA256 with per-user salt; 256-bit bearer tokens, only SHA-256(token) stored; 12-hour sessions.
- **Face:** InsightFace buffalo_l (SCRFD + ArcFace 512-d). Sign-up stores 11 templates (3 frontal + 8 head directions); login adds a random head-turn challenge; match threshold 0.45; one face, one account. OTP is the fallback when no camera is available.
- **ID card:** RapidOCR (PaddleOCR Chinese models on onnxruntime) reads the PRC ID; code checks the GB 11643 checksum, age ⩾18, expiry and the owner′s name.
- **Consent:** three separate, unticked boxes (credit inquiry, sharing with partner banks, face and financial data). The seller signs the canonical consent document with **Ed25519**; it expires after 90 days and can be revoked. Agent 1′s data tools, rule LAW-01 and Agent 4 all call the same check.

- **Shop ownership:** a loan dossier can only be created from a connected shop: marketplace OAuth (Taobao/Tmall, JD, Pinduoduo, Douyin, Kuaishou) or a code sent to the shop's registered phone (hashed, 5 minutes, locked after 5 failures).

Integrity and regulation

- Local ledger blocks are HMAC-keyed; outside dev/test a missing or default `CREDIT_LEDGER_KEY` stops start-up (fail closed).
- WeBank webhooks (`/inflows` , `/outflows`) carry an HMAC-SHA256 signature and are idempotent on transfer id.
- Rate ⩽24% a year (LAW-02); large-transaction reporting thresholds 2,000,000 (organization) / 500,000 (individual); prohibited, licensed and counterfeit goods detected from item names and descriptions; individual sellers warned at 80% of the 100,000 CNY registration threshold.
- Every rejection states the rule applied, the main factor and how to improve (LAW-03, in line with PIPL Article 24 on automated decisions).

8. API and user interfaces

One FastAPI service exposes about 120 REST endpoints under `/v4` , documented in `BackEnd/API_config/API_CONTRACT_V4.md` (every route, request, response and error code) and interactively at `/docs` . Every response carries an `x-request-id` that matches the JSON log line.

Area	Main routes	Used by
Applications (23)	create, `/cccd` , `/identity` , `/consent/draft` , `/consent` , `/plan` , `/statement-images` , `/verify` , `/answers` , select, sign	Seller wizard, Agents 1–3
Loans (14)	schedule, `/minimum` , `/prepay` (both need explicit confirmation by OTP or face), simulation	Seller, Agent 4b
Shops (9)	OAuth connect per marketplace, registered-phone code, demo shops	Seller
Chain (13)	anchors, case verification, withdrawals, decrypt, bank catalogue sync, node status	Officer, admin, banks

Area	Main routes	Used by
Banks, payouts (11)	parallel decisions, withdrawal pieces, on-chain terms	Partner banks
Officer, admin, post-audit (13)	queues, flag resolution, payout re-authorization, parameter proposals and approval, autopilot status	Staff
Face, auth (11)	enroll, login frames, register, login	All users
Complaints, notices, reports, legal (16)	complaint desk, notification center, monthly/quarterly/annual reports, statute search	Seller, officer
WeBank webhooks (2)	signed `/inflows`, `/outflows`	WeBank core banking

Six static pages are served from the same process (`Frontend/giaodien`), with plain JavaScript and no build step:

- **Landing, login, register:** face enrollment at sign-up, face or password login.
- **Seller workspace:** an 8-step wizard (borrower type → shop connection → ID card → consent and signature → plan → verification → offer → contract), chat with Agent 2, loan page with schedule and repayments, notices, reports, "My bank".
- **Officer / admin console:** dossier list, per-agent tabs, a "Full flow" tab that replays B1–B9 with the formula and the real numbers and checks them against the stored sheet and its on-chain seal, chain and ledger status, Agent 4/5 autopilot panels.
- **Bank portal:** parallel-approval queue, the bank's decryptable withdrawal pieces, on-chain terms.

The interface is in Vietnamese for the team's demo; the API and data model are language-neutral.

9. Testing, evaluation and deployment

BuoyUp is measured by two kinds of tests: functional tests that check every number and state transition, and an adversarial TestBench in which every case is an attack and **PASS means the system resisted it**. All suites run offline on a fresh temporary database with the LLM off and the chain forced to local mode, so they never write to the team's FISCO chain.

Test suites

Suite	Scope	Result
Backend unit and integration (pytest)	Algorithm B1–B12, agents 1–5, contract, syndicate, chain and withdrawals, consent signatures, face and OTP, shop connection, seller services	467 passing
TestBench · security cases	Login, face and sessions (16), credit tampering (2), LLM prompt injection and legal citations (4), multi-bank authorization and input validation (9)	31 / 31 attacks resisted
TestBench · 50 end-to-end loan dossiers	Honest sellers, false declarations and flow abuse through the real API (below)	50 / 50 resisted

The 50-dossier TestBench

The 50 sellers are built from four reference dossiers (fashion, cosmetics, groceries, electronics) and run through the real API, including ID card OCR and the signed declaration. Each has a difficulty from 1 to 5 and one of three kinds:

Kind	Cases	Examples	Round 1 (17/10, before fixes)	Round 2 (17/10, after fixes)
Honest seller	10	standard household business, personal seller, thin FATE, under 12 months	9	10
False declaration	26	inflated price or need, prohibited goods under another category, fake statements, forged or expired ID, minor, counterfeit goods, related-party supplier, laundering through the WeBank account, misuse of funds	24	26

Kind	Cases	Examples	Round 1 (17/10, before fixes)	Round 2 (17/10, after fixes)
Flow abuse	14	edited amount/tenor/method, skipped declaration, loan stacking, prompt injection in chat, payee swap, double disbursement, IDOR, privilege escalation, junk uploads	14	14

Round 1 found three real weaknesses, each fixed in code and parameters:

1. **DS-07 cold-start trap:** a small thin-FATE seller could never reach the minimum loan, so never build history. Fix: the seed ratio never drops a reachable limit below the product minimum.
2. **DS-29 inflation within tolerance:** typed revenue up to 15% above tax earned the statement bonus. Fix: the bonus now requires screenshots that show the typed figures (`revenue.bonus_requires_images`).
3. **DS-40 counterfeit goods without a brand field:** after the brand box was removed from the form, counterfeits slipped through. Fix: IP signals are read from the item name and description.

Each dossier makes about 7 API calls and completes in 0.75 s on average.

Other evidence

- **API contract check:** 9 demo scenarios replayed against the contract (`Frontend/tests/v4-api-check.py`); UI checks in Node (`v4-ui-check.mjs` , `c1-check.mjs`).
- **Post-loan services log:** an 8-step scripted run writes a JSON and Markdown evidence log (`TestBench/evidence/seller_services-20261017-2316.*`); its final check confirms zero repayments created by Agent 5 and zero capability violations.
- **RAG quality:** recall@5 0.82 on a 22-question hold-out set (Section 4).
- **On-chain proof:** in remote mode each loan card shows the block number and 4/4 node confirmations of its disbursement; the officer console verifies the stored sheet against its on-chain seal.

Deployment

- **Docker:** `cp BackEnd/API_config/.env.example BackEnd/API_config/.env` , fill it, then `docker compose up -d --build` ; the container runs as a non-root user (uid 10001) with data in the `buoyup-data` volume and a health check on `/healthz` .

- **Without Docker:** `./run-local.sh` (creates the virtualenv, serves on port 8000); `./run-local.sh test` runs every suite.
- **Requirements:** 1 vCPU / 1 GB RAM minimum, no GPU (the LLM runs on a separate vLLM host). HTTPS through Nginx and Certbot, using the template in `deploy/`.
- **Operations:** structured JSON logs, nightly SQLite backup (`deploy/backup.sh`), demo reset that archives rather than deletes on-chain dossiers. Full guide: `HANDOVER.md`.

10. Known limitations and next steps

BuoyUp is a pilot-grade system, not yet a lending product for real money. The gaps below are known and tracked; each has a defined path to close it.

Limitation	Impact	Next step
Seller data sources (shop, registry, cash flow, tax, credit) and FATE use sample data	Limits are correct for the sample profiles, not proven on real sellers	Replace functions in `data_sources/sources.py`; agents and algorithm stay unchanged
Challenger model trained on synthetic labels	Agent 3′s risk score is illustrative	Retrain with `scripts/train_challenger.py` on banks′ repayment labels
Many parameters are hand-set (`DAT_TAY`)	Thresholds not yet validated by outcomes	Agent 5 backtests and proposes; dual-admin approval
Chain service exposes demo keys and has no API token; 4 nodes run on one machine	"No single party can alter" does not yet hold	Disable demo keys, set `apiToken`, banks hold their own keys, nodes run by separate parties
SMS OTP generated and checked in the browser	OTP proves nothing about the phone number	Server-side `/otp/send` and `/otp/verify` with one-time proof once an SMS provider is connected
Face check is a head-turn challenge, not certified anti-spoofing	A pre-recorded video could pass	Use face as a second factor or add a liveness model
SQLite, one process, no schema migrations	Fine for demo and small pilot	Move to Postgres; run autopilot as a separate worker
Marketplace OAuth connectors need API keys	Only demo shops can be connected today	Register apps with each marketplace and finish the `TODO(<platform>)` items

Limitation	Impact	Next step
Legal open points (FATE licensing, platform advance-and-recover, bank self-approval, 30% minimum share)	Need confirmation from counsel and partner banks	Team decision with legal review

The architecture already isolates each of these behind one module or one parameter, so closing a gap does not touch the agents' contracts or the algorithm.